0计算机基础的Agent应用与开发入门 · Lesson 2

Lesson 2: What Software Is Made Of — Programs, Data, Modules, Responsibilities, and Flow

> say what a piece of software is made of; tell programs and data apart; split a large processing job into modules and describe each module's responsibility; recognize sequence, branching, and loops in a program; and draw the structure of a small piece of software.

Author: Ritten_poem

> **In this lesson you will learn to:** say what a piece of software is made of; tell programs and data apart; split a large processing job into modules and describe each module's responsibility; recognize sequence, branching, and loops in a program; and draw the structure of a small piece of software.

1. Start with an example you use every day

You open a map app on your phone, type "convenience store nearby," and a list of shop names appears, each with a distance and opening hours. You tap the first one. It draws a route and tells you it is a 12-minute walk.

You did two things: search, and pick one. But in those few seconds the software did far more. First it had to work out where you are standing, because it cannot calculate a distance without that. Then it had to search millions of business listings for the ones whose names match "convenience store." Then it checked them one by one to see which are open right now. Then it sorted the ones still open by distance, nearest first. Then it drew the result as a list you can read.

None of that happens in one step. It is split into blocks, and each block does one job. In this lesson we open that app up and look at what it is actually made of.

2. Why ask what it is made of

In the last lesson we broke a request down into input, processing, output, and data. One question was left open. The processing step said "calculate the distance," "keep the ones that are open," and "sort them." That sounds as though the software had thought it through. It hasn't.

Processing is possible only because those rules are actually written down somewhere. Where they live, whether they can be checked on their own, and whether changing one feature disturbs another all depend on how the software is organized.

This matters most when you work with a Code Agent. When it hands you two hundred lines of code that fail with an error, you need to be able to ask one meaningful question: is the rule itself wrong, is it in the wrong place, has the wiring between modules broken, or is the data itself at fault? If you don't understand how software is put together, all you can do is paste the error back and watch it change things at random.

3. Four things make up a piece of software

Component	What it is	In one line
Program	A written set of rules and steps	Instructions to follow
Data	The things the program works on	Raw material to process
Module	A small piece the big program is cut into	Bricks, each with one job
Responsibility	What this module does, and does not do	The job description

Here is how they relate: **the program is the rules, the data is the raw material the rules act on, the program is cut into modules, and each module carries one responsibility.**

In the last lesson we said data is the part people forget most often. This lesson has a warning running the other way: **programs and data are two different things, and they can be stored separately.** The menu price is data. "The total is the sum of the item prices" is a program. You can change a price without touching the program, and change a promotion rule without touching the data. Once you can tell these two apart, you know which one to change when a requirement changes.

4. Taking each part apart

1. Programs: a very literal list of instructions

A program has no intuition and does not "use its judgment." It is a list of steps: do this, then do that, and when step 3 is done, jump to step 4.

Open any piece of code and this is what you are looking at. The programmer's job is to turn something vague into instructions so explicit that no thinking is required. "Work out the total" is far too vague for a program. What it needs is:

1. Take the price of the pearl milk tea;
2. Take the price of the coconut jelly;
3. Add the two numbers together;
4. Save the result.

One property of programs is worth remembering: they execute strictly in the order they were written, skipping nothing and doing nothing extra. You may expect it to "understand what you meant." It only understands the line that was written.

> **A common mistake:** imagining a program as one single thing. In practice a piece of software usually contains many program files — one that calculates distance, one that reads data, one that draws the screen. They are stored apart and put to work together.

2. Data: the raw material

Data is what the program processes. In "finding a convenience store," the data includes:

- **Shop listings:** name, address, coordinates, opening hours, whether it is open now;
- **Your location:** the coordinates the phone gets from GPS or location services;
- **The search term:** the three characters you typed;
- **Filters:** the boxes you ticked for "open 24 hours" or "has parking";
- **Computed results:** 800 meters away, a 12-minute walk.

The point about data is that it does not do anything. It gets looked up, compared, and calculated.

A good test for identifying data: if you change its value and the program's *method* stays the same while only the *result* changes, it is data. Switch "open 24 hours" to "still open after 8" and the program does exactly the same work; only the set of qualifying shops changes.

> **A common mistake:** confusing data with the screen that displays it. "800 m" is data. Whether it is rendered smaller or in a lighter shade is the interface. When you change a requirement, say clearly which one you mean.

3. Modules: cutting a big program into bricks

If every rule were written into one file, that file would grow until nobody dared to touch it. So programmers cut the code up by function, and each piece is called a **module**.

"Finding a convenience store" can be split roughly like this:

Module	What it is responsible for	What it is not responsible for
Location	Knowing where you are now	What you are looking for
Search	Finding candidate shops matching "convenience store"	Deciding which is closer
Data	Reading shop records and reporting how many exist and which are nearby	Doing the filtering
Filtering	Keeping the shops that are open now	Calculating distance
Sorting	Ordering them by distance, nearest first	Drawing anything
Interface	Rendering the list and the map	Recalculating anything

The benefit of modules comes down to one sentence: when something goes wrong, you know who to ask.

Previously, if the order was wrong, you would hunt through a mass of code. Now ordering is the sorting module's responsibility, so you can start there. If the distance values themselves are wrong, check the data or calculation that produced them. Clear responsibilities help distinguish bad input data from a bad sorting rule.

> **A common mistake:** cutting modules by screen. You might divide the whole app into a "home screen module" and a "detail screen module." The problem is that the same rule for calculating a price may then be written twice, once in each module, and you will change one and forget the other. **Cut by responsibility, not by screen.**

4. Responsibility: the job description

A module's **responsibility** is its list of what it should do and what it should not do. It is the most important sentence in a piece of software, and the one most often skipped.

In the table above, the second column of each row is what it does, and the third is what it doesn't. The third column gets skipped constantly, and it is the part that makes the division of labor work.

Three questions to test whether a responsibility is clear:

1. Can you state it in a single "only … not …" sentence? For example: "only reads shop records, does not filter."
2. If tomorrow's requirement changes, does this module need to change? If not, its responsibility is clean.
3. Does its name summarize what it does? "Sorting module" does. "Utility module" does not.

> **A tip:** a module that does three things at once usually wants to be split up. While you split it, you will often discover that the requirement was unclear in the first place — that is a bonus you get for free.

5. Flow: how a program moves forward

Every program moves forward in only three ways. Learn these three and you will know what any piece of code is doing.

Sequence: finish one thing, then do the next. "Read the shop data → calculate distance → sort → draw." Top to bottom, with no forks.

Branching: go in different directions depending on a condition. "If this shop is closed, skip it. If every shop is closed, show 'No convenience stores open nearby.'"

Loops: repeat the same action until a condition stops holding. "Take the candidate shops one at a time and check whether each is open." After the first there is a second, after the second there is a third. That is a loop.

Why loops matter so much: there might be ten thousand shops, and you cannot write ten thousand lines of code. You write one loop and let it do the repeating for you. This is the idea of "a rule instead of repetition," and it is exactly what "processing" really looks like in practice.

Drawn out, these three ways look like this:

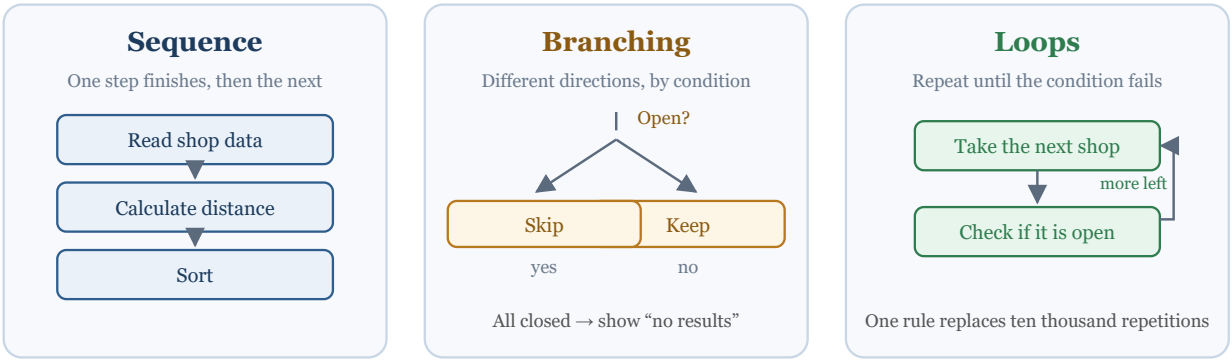


Figure 2-1. A program has only three ways to move. Every complicated program is these three, combined again and again.

5. How modules work together

Once the modules are cut, the next question is how they find each other. That is what **calls** mean: a module finishes its part and hands the result to the next one.

The chain for “find a convenience store” runs like this:

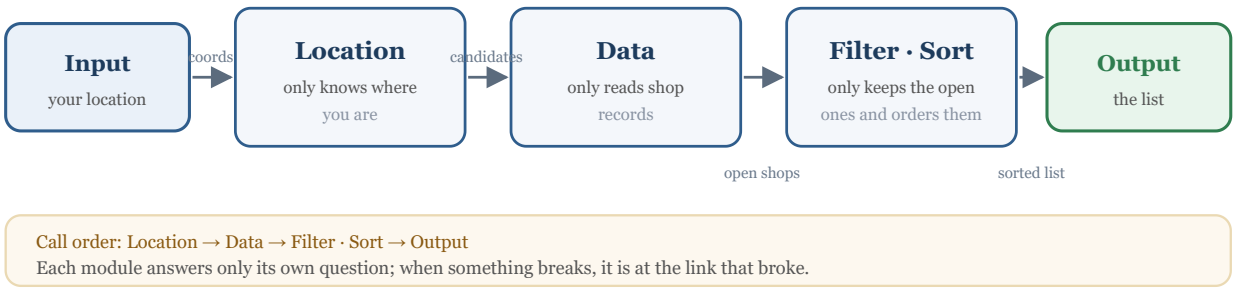


Figure 2-2. What each module does and how they work together. At the top, the responsibility of each module; below, the order in which they are called. Each module answers only its own question.

Read it along the arrows: the location module hands over “where you are,” the data module hands over “the candidate shops,” the filtering module hands over “the shops that are open,” the sorting module hands over “the sorted list,” and the interface module finally hands over “the screen you see.”

Once you can read that diagram, you have a skill: given an unfamiliar piece of software, you can ask what every arrow means. When something breaks, you can say which link failed, instead of saying vaguely that “it calculated wrong.”

6. Drawing the structure of a piece of software

You need no special tool — a sheet of paper is enough. Draw it in four steps:

1. **Draw input and output first.** The leftmost box is what the user provides; the rightmost is what the user sees. You already know this from the last lesson.
2. **Then cut up the “processing” in the middle.** Divide it by responsibility into modules, one box each, named with a verb plus an object: “sort,” “filter.” Avoid names like “tools” or “features.”
3. **Connect them with arrows.** The arrow direction is the direction the data moves, and label each one with what exactly is being passed.
4. **Check two things at the end.** Is there a module doing two unrelated jobs? Is there an arrow that does not say what it carries?

> **A common mistake:** rushing to draw the interface while drawing the structure. The diagram should contain words like “input,” “filter,” and “output,” not “home screen” or “button.” Drawing the screen is something you get to later.

7. Three common mistakes

Mistake 1: putting everything in one place. Location, filtering, and sorting all in a single file. It runs, but you cannot check it and you cannot safely change it. “It runs” and “it can be maintained” are two different things.

Mistake 2: naming modules too broadly. Names like “utility module,” “common module,” and “management module” amount to no division of labor at all. If you cannot find an accurate name, the responsibility has not been thought through yet.

Mistake 3: writing what a module does, but not what it doesn’t. If every module says “responsible for handling data,” everyone is responsible and no one is. Division of labor lives in the boundaries, not in the scope.

8. Hands-on practice

Three exercises, from easiest to hardest. Try each one yourself before you check the sample answers at the end.

Exercise 1: tell programs from data (warm-up)

Below are several sentences from an expense-tracking app. Sort them into “program” and “data”:

- Every entry contains a date, an amount, and a category;
- A report for last month is generated automatically on the 1st;
- Coffee, 25;
- When spending exceeds 3,000, show a reminder;
- The shop’s default category list;

When you are done, ask yourself two more questions: if “coffee, 25” becomes “coffee, 32,” which column did you change? If “a report is generated on the 1st” becomes “a report is generated on Monday,” which column is that?

Category	Sentences I sorted here
Program	
Data	

Exercise 2: cut a piece of software into modules

Below is a request for a reading-notes app, written as seven features. Cut it into modules, each doing only one thing, and write down the responsibility of each module using the pattern “only … not …”.

> ① The user can enter a book title, an author, and the page they reached; ② A note can be attached to a specific page of a specific book; ③ The home screen lists books by most recently read; ④ You can give a book a star rating; ⑤ Notes can be searched; ⑥ Search results should show which book each note came from; ⑦ After finishing a book, you want to archive it.

Goal: produce four to seven modules. If a module is hard to name, check whether its responsibility is clear. Different module counts are acceptable when the division of work is justified.

Module name	It is only responsible for…	It is not responsible for…

Exercise 3: draw a structure diagram

Choose an app you use every day — the one you broke down in Exercise 1 of the last lesson, or any other. Draw its structure following the four steps above, and meet these requirements:

1. Draw at least four modules;
2. Name each module with a verb plus an object;
3. Label every arrow with what it carries;
4. Mark one place each where sequence, branching, and a loop are used.

When you finish, check your diagram against this list:

- [ ] Every module's responsibility can be stated as "only … not …";
- [ ] Every module's name summarizes what it does;
- [ ] Every arrow says what it carries;
- [ ] No module is doing two unrelated jobs at once;
- [ ] Someone who has never seen this app could follow the diagram and describe its complete flow.

Challenge (optional)

Find a feature description for an existing app — the screenshots and description in an app store are enough — and break it into modules and arrows using only that text. You will notice that descriptions are written in terms of *features*, while a structure diagram is drawn in terms of *responsibilities*. A translation sits between them, and that translation is exactly the skill you will need most when writing a brief for a Code Agent.

Sample answers

Exercise 1

Category	Content
Program	A report for last month is generated automatically on the 1st; when spending exceeds 3,000, show a reminder
Data	Every entry contains a date, an amount, and a category; coffee, 25; the shop's default category list

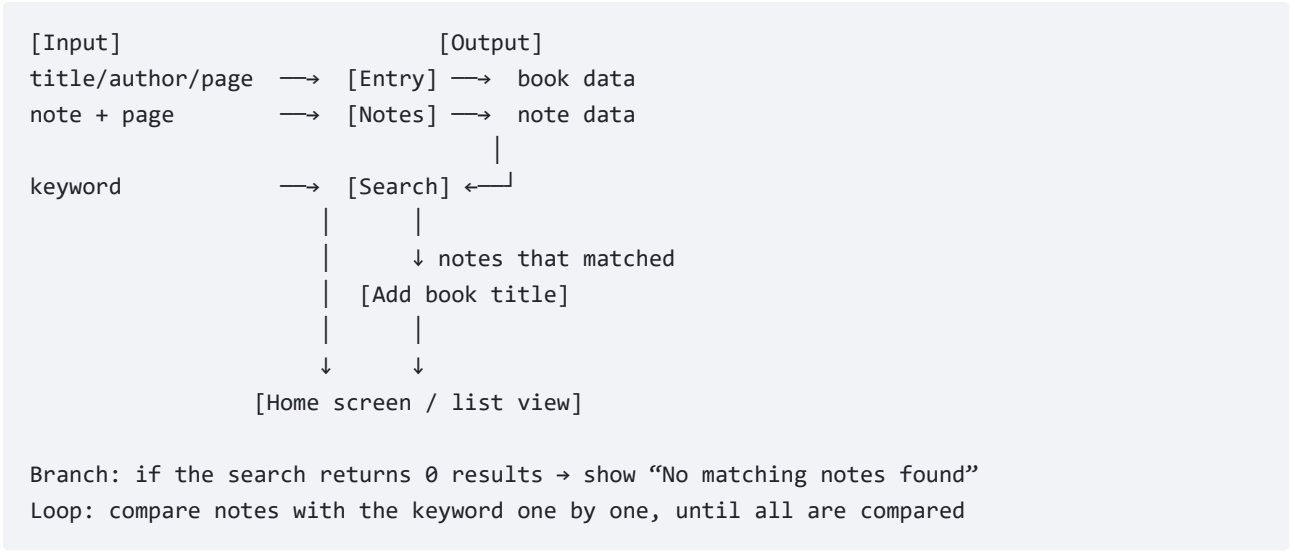
Turning "coffee, 25" into "coffee, 32" changes **data**. Changing the report from the 1st to Monday changes the **program**. The pattern is useful: knowing which column a change lands in tells you which part of the files you ask a Code Agent to touch.

Exercise 2 (one possible split; yours will differ)

Module name	It is only responsible for…	It is not responsible for…
Entry	Receiving and saving book details	Displaying, and searching
Notes	Saving note text, page number, and which book it belongs to	Deciding what appears on the home screen
Home	Deciding which books the home screen shows	Reading note contents
Rating	Recording and changing the star rating	Changing the notes themselves
Search	Finding notes matching a keyword	Adding the book title to the result
Result display	Showing the book name on search results	Finding the notes
Archive	Changing the status of a finished book	Deleting notes

Look at the last two rows. Requirement ⑥ is very tempting to fold into the search module, but its job is **display**, and the job of finding is a different one. Squeeze it in and that module is now doing two things. Places where you cannot name a module are places where the request mixes two responsibilities.

Exercise 3 (sample, using the reading-notes app)



Checked against the list: all five modules have stated responsibilities; “add book title” is its own module precisely because it only displays; every arrow says what it carries; and sequence, branching, and a loop are each marked once.

Checkpoint: after this lesson, you should be able to…

- Say that a piece of software is made of programs, data, modules, and responsibilities, and explain how they relate;
- Decide whether a sentence is program or data, using the test “does changing it change the program’s method?”;
- Split a large processing job into modules, stating each one as “only … not …”;
- Find sequence, branching, and loops in an unfamiliar description;
- Draw a structure diagram for a small piece of software and say what every arrow carries.

Next lesson

Lesson 3, *How Software Connects*, asks whether all these modules and records live on your phone. If some work happens on another computer, how does the phone send a request and receive a result?

We continue the convenience-store example to explore networks, clients and servers, requests and responses, and APIs. Lesson 4, *From Design to Running*, then connects the diagrams from Lessons 1 to 3 to a small example that actually runs. Lesson 5 explains files, environments, dependencies, and operating systems. This sample includes Lessons 1 and 2; later lessons are provided separately.